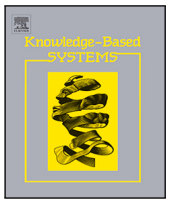
Knowledge-Based Systems 325 (2025) 113848

Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys



DTIU: A self-supervised grid-enhanced diffusion model for trajectory imputation in unconstrained scenarios

Zhijing Hu [a], Hao Yan [b], Kuihua Huang [a], Jincai Huang [a], Zhong Liu [a], Changjun Fan [a,*]

[a] Laboratory for Big Data and Decision, College of Systems Engineering, National University of Defense Technology, Changsha, Hunan, China
[b] School of Computer Science, Central South University, Changsha, Hunan, China

ARTICLE INFO

Keywords:
Diffusion model
Trajectory imputation
Unconstrained scenarios

ABSTRACT

While the widespread of location-based services has led to a proliferation of trajectory data, it is often accompanied by the inevitable problem of incomplete data due to various reasons, e.g., sensor failure and privacy concerns. Imputing the incomplete trajectory is critically important to a range of practical applications, e.g., traffic management and emergency response. Most recent proposals on trajectory imputation are designed for the urban scenario where a road network is available, which may not work well in the unconstrained environment scenario. As there are no road networks or predefined paths in the unconstrained environment, existing approaches for trajectory imputation may result in insufficient input data and thus lead to sub-optimal performance. To address this issue, we propose a self-supervised grid-enhanced **D**iffusion model for **T**rajectory **I**mputation in **U**nconstrained scenarios (DTIU). DTIU includes a diffusion model specifically designed for trajectory imputation tasks. DTIU avoids the problem of insufficient input by using trajectory data as the only input. To contend with missing position information and effectively learn the spatio-temporal correlations, we design a GridFormer based on AutoEncoder with an adaptive grid information enhancement strategy. DTIU adopts a self-supervised training strategy inspired by masked language models, aiming at address the data sparsity issue. Extensive experiments on real data offer insight into the effectiveness of the proposed framework. This marks the first application of diffusion models to tackle trajectory imputation in unconstrained scenarios.

1. Introduction

The collection and utilization of location data recently have become ubiquitous, generate increasingly massive amounts of trajectory data [1], coupled with problems of data loss. Due to device failure, human error, and other unforeseen reasons, Data loss is inevitable, and can be significantly detrimental to the performance of downstream applications. Thus, the data imputation task, which involves filling in the missing data points in a dataset, is crucial to maintaining the reliability and usability of trajectory data [2,3]. To this aim, this paper focuses on the problem of trajectory data imputation, which aims to predict and fill in missing values in the trajectory data. Specifically, this task typically gives a segment of incomplete trajectory data including longitude and latitude as input. And then the associated imputation model is required to impute the trajectory data at the missing locations.

The existing trajectory imputation methods mainly include traditional statistical methods and deep learning methods. Efforts have been made in trajectory data imputation methods which are typically based on time series data imputation, including approaches based on traditional statistics [4,5] and convolutional and recurrent neural networks [6,7]. For trajectory imputation, researchers have designed multiple attention mechanisms to capture periodic patterns of human movement [3,8]. Additionally, graph neural networks (GNNs) have been utilized for trajectory recovery, leveraging their ability to model spatial dependencies [9,10]. There are also numerous works focusing on trajectory representation learning (TRL), which aims to learn latent representations of trajectories that capture their essential characteristics [2,11].

Existing trajectory imputation models are mostly designed for urban traffic scenarios, and they may not work well in the unconstrained environment scenarios. As shown in Fig. 1, unconstrained environment scenarios usually refer to scenarios where the subject does not follow fixed routes, such as road networks or road status. In practice, trajectory are often based on unconstrained environment scenarios, especially at sea or in wild environments outside cities. Existing methods rely on additional input of road information, but in unconstrained environment, they cannot obtain effective input without this information. Directly applying models based on urban scenarios to unconstrained

* Corresponding author.
E-mail address: fanchangjun09@163.com (C. Fan).

https://doi.org/10.1016/j.knosys.2025.113848
Received 21 October 2024; Received in revised form 15 May 2025; Accepted 24 May 2025
Available online 11 June 2025

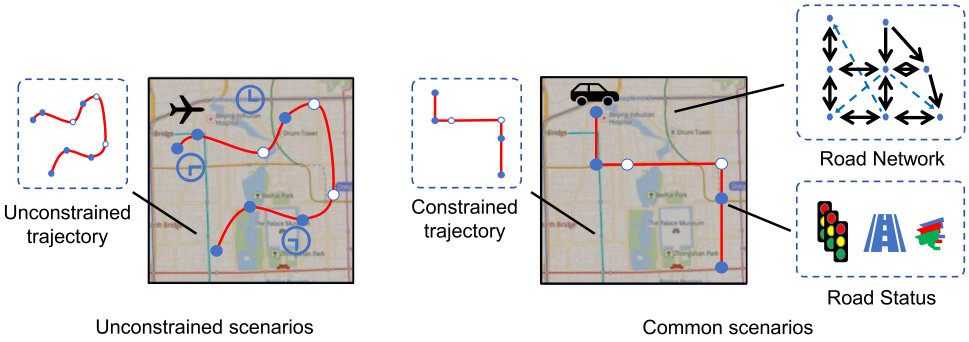


Fig. 1. Trajectories in different information scenarios. In common scenarios, trajectories are often accompanied by valid road surface information input, such as road network or road status. However, many models perform poorly in unconstrained scenarios environment because they cannot access this information. The lack of structured auxiliary information exacerbates the challenge of accurately inputting missing trajectory data.

environment scenarios may cause remarkable performance degradation. Therefore, we need a novel trajectory imputation model that can efficiently complete existing tasks while also being able to adapt to a variety of different scenarios, especially in unconstrained environment scenarios.

However, it is non-trivial to develop this kind of model, due to the following challenges. First, the imputation task is more difficult in unconstrained scenarios compared to common scenarios,. When considering the unconstrained scenarios, the trajectory is more complex and variable. And lack of road information makes it more difficult for existing models to capture the pattern features of trajectories. Existing trajectory imputation models mainly target at trajectory data in urban traffic scenarios. These traffic trajectories usually benefit from the road network information and constraints, making the imputation task relatively easier. In contrast, it is difficult for these existing models to deal with trajectories without a priori information in unconstrained environment scenarios. Examples include bird flight trajectories, human walking trajectories, and low-altitude unmanned aerial vehicle (UAV) trajectories. These trajectories lack auxiliary information, as shown in Fig. 1. Specifically, we consider that any additional data information other than the trajectory data itself can be considered as auxiliary or a priori information. The lack of predetermined paths and constraints in unconstrained environment scenarios greatly increases the difficulty of the task by requiring the model to infer movement patterns without the help of external spatial cues. So far, there is still a lack of promising solutions to these imputation challenges [12]. Therefore, the imputation task is more challenging. Second, the spatio-temporal dependencies of trajectories are complex and difficult to be well captured by existing models. Trajectory data is a special type of spatio-temporal data with specific positional relationships and long-range dependencies in time series modeling. Previous methods, such as GNNs, are more suitable for capturing static relationships but may struggle with dynamic spatial and temporal dependencies [1]. In addition, trajectories in such unconstrained environments are similar to crowd flows, and spatial correlations are complex and sometimes do not follow spatial smoothness, requiring the fusion of multiple semantic information to extract spatial relationships [13]. Therefore, a new method is required to capture these complex dependencies more efficiently. In addition, trajectory data formats are inconsistent and the amount of valid data is small. The variety of data in the trajectory data domain has led to a lack of uniform data standards in the field due to privacy or commerciality. This often lacks effectively labeled trajectory data for imputation tasks, limiting the conditions for supervised learning. Therefore, new training methods need to be developed to capture the intrinsic patterns of the data.

To address the aforementioned challenges and the limitations of existing models, we propose a novel method named DTIU, which stands for self-supervised grid-enhanced trajectory imputation diffusion model in unconstrained scenarios. This method is designed for unconstrained environment scenarios to impute missing trajectory data. To effectively capture the dynamic spatial dependencies crucial for accurate trajectory imputation, we employ a Markov-chain-based diffusion model as the core framework. This model is particularly suited for unconstrained environments where constraints are minimal, and only trajectory data is available as input. The diffusion model is able to adaptively capture these dependencies and enhance the robustness of the trajectory imputation process. We also introduce a GridFormer module to handle the second challenge, which is designed to enhance the specific positional relationships and long-range dependencies inherent in trajectory data. This module complements the diffusion model by providing a more nuanced understanding on the spatial characteristics of the trajectory data. We consider that the integration of the GridFormer with the diffusion model represents a novel approach in the field of trajectory imputation, as it combines the strengths of both probabilistic and deterministic modeling techniques. Furthermore, to overcome the last challenge, posed by the limited availability of trajectory data and the absence of labeling, we develop a self-supervised training method inspired by masked language modeling. This method involves dividing the raw trajectory data into conditional inputs and estimation targets, which allows the model to learn effectively from unlabeled data. By doing so, DTIU can leverage the available data more efficiently, improving its performance in imputing missing trajectory information. In summary, our main contributions are as follows:

- We propose the DTIU model for trajectory imputation in unconstrained environment scenarios, addressing the unique challenges posed by the lack of structured auxiliary information. DTIU integrates a diffusion model with the GridFormer module to effectively capture complex spatio-temporal dependencies.
- We introduce the GridFormer, which enhances the DTIU model's ability to handle trajectory data by providing a grid-based representation that preserves positional relationships and long-range dependencies.
- We develop a self-supervised trajectory imputation training method inspired by masked language modeling. This method allows the model to learn from raw trajectory data without the need for extensive labeled datasets, making it suitable for scenarios with limited label.
- We provide extensive experimental evidence demonstrating the effectiveness of our approach. Numerous experiments conducted across four different datasets showcase the strong competitiveness of DTIU and its superiority in achieving accurate trajectory imputation. Our model outperforms existing methods, particularly in unconstrained environment scenarios where traditional models struggle.

2. Related work

Missing values are common in trajectory data and multivariate time series data. To fully utilize and analyze them, the related research has become a very rich topic.[14,15] We will review related work in three areas: trajectory representation learning and trajectory imputation, deterministic imputation models, and probabilistic imputation models, as our paper is deeply related to these three parts.

2.1. Trajectory representation learning and imputation

How to impute trajectory has always been an important issue in spatiotemporal data analysis and management, with most work modeling using sequence learning models. Zerveas et al. [2] used transformers to model trajectory sequences. Xia et al. [3] modeled the periodicity of mobility learned from history with recurrent neural networks (RNNs), and Sun et al. [8] designed various attention mechanisms to capture the periodic patterns of human movement. Additionally, Jeon et al. [10]; Liang et al. [16]; and Sun et al. [8] also utilized graph neural networks(GNNs) to capture complex locational transition patterns for trajectory recovery. Some two-stage methods first use GNNs to learn road representation vectors and then transform them into trajectory representations by using deterministic models.[17,18] Huang et al. [11] also designed a variety of self-supervised tasks and trained using contrastive learning. Liu et al. [19] proposed a non-autoregressive multi-resolution sequence imputation model to establish non-autoregressive correlations between missing trajectories and temporally adjacent observations. Yang et al. [20] used LSTM encoding and decoded using hidden features for trajectory imputation for each object. However, they are only suitable for scenarios with strong information and do not perform well in unconstrained environment scenarios.

2.2. Deterministic imputation models

Early time series imputation saw some representative autoregressive models, such as ARIMA [21], VAR [22] and MICE [23] employs chained equations to impute missing values, and Hudak et al. [24] propose K-nearest neighbors algorithm which uses the average of neighboring values to fill in missing data. Deep learning models that emerged later can capture the temporal dependencies of time series and have shown better performance. Deterministic models perform representation learning and modeling of sequences to generate determined imputations given observed values. Berglund et al. [25]introduced bidirectional recurrent neural networks to effectively estimate missing values. Che et al. [26] and Cao et al. [27] replace the missing values in the input sequence with specific markers. To enhance the capability of representation learning, Choi et al. [28] applied a self-training mechanism to multivariate imputation, and Shukla et al. Suo et al. [29,30] proposed attention-based multivariate imputation models, considering the inter-variable correlations. Du et al. [31] employed self-attention to capture time features and feature correlations. Additionally, there have been models for time series imputation using differential equation networks, such as the controlled differential equations networks [32], proposed by Morrill et al. and the Latent ordinal differential equations networks, [33], proposed by Rubanova et al. Moreover, Cini et al. [34] and Chen et al. [35] proposed combining RNNs with Graph Convolutional Networks (GCN) to produce more accurate imputations.[14]

2.3. Probabilistic imputation models

Given the importance of accounting for uncertainty in various real-world scenarios [15,36], there has been an increasing effort focused on probabilistic methods in recent years. Early attempts at probabilistic models for Missing Data Imputation expanded Generative Adversarial Networks (GANs) [37] to multivariate imputation, which resulted in a proliferation of GAN variants each addressing different aspects of the problem, such as E2-GAN [38], GRUI-GAN [39], and SSGAN [40]. In the other hand, Dalca et al. [15] and Fortuin et al. [41] based their temporal imputation models on Variational Autoencoders (VAEs) to enhance the interpretability of the models. Recently, diffusion models [42,43] have shown impressive performance in various generative tasks, such as audio and image synthesis. Tashiro et al. [1] attempted to learn the conditional distribution of diffusion models based on conditional scores [44,45] by feeding observed values into the denoising module of diffusion models. Alcaraz et al. [15] employed structured state space models as its internal model architecture. Wang et al. [14] takes into account the correlation and ensuring consistency between observed values and missing values. However, within the domain of spatio-temporal data, there is a dearth of appropriate probabilistic models dedicated to the task of trajectory imputation. Additionally, models that treat trajectory data merely as multi-dimensional time series imputation tasks fail to adequately learn the distinctive relative spatial relationships and the long-term dependencies inherent in spatiotemporal data. This has been identified as an existing issue in the introduction.

In summary, these above methods either neglect the spatiotemporal data characteristics of trajectory data, treating it merely as multivariate time series, or are based on road networks proposing various spatial correlation construction methods. Therefore, these models are only applicable in scenarios with strong information and imputation for deficient information scenarios remains an existing issue with current models still needing improvement.

3. Preliminary

In this section, we focus on the problem of trajectory sequence imputation in unconstrained environments. Our goal is to estimate the missing trajectory values based on the observed values of these sequences. The trajectory sequences we are interested in contains several sequences of the same length, each of which contains missing values. We here formally define the problem of trajectory imputation.[14]

Definition 1. Trajectory. In this paper, we consider that processed trajectory sequences containing N trajectories of equal length L can be represented as $\boldsymbol{X}$. We represent the ith trajectory as $\boldsymbol{X}^i \in \mathbb{R}^{L\times 2}$, and all trajectories can be represented by $\boldsymbol{X} = \{X^1, X^2, \ldots, X^i, \ldots, X^N\}$. And we only consider the longitude and latitude values $x^{i,j}_{\text{lon}}, x^{i,j}_{\text{lat}}$ for each trajectory point $x^{i,j}$, assuming that the longitudinal and latitudinal data of trajectory points are correlated. In addition, we denote the trajectory of the t-th step of noise addition by $\boldsymbol{X}_t$. In particular, we denote the trajectory $\boldsymbol{X}$ without noise addition by $\boldsymbol{X}_0$, as the 0-th step of noise addition. $\boldsymbol{X}^{\text{cond}}$ or $\boldsymbol{X}^{\text{cond}}_0$ is the observable trajectory data, $\boldsymbol{X}^{\text{miss}}$ is the missing trajectory data. In Chapter 4, we take $\boldsymbol{X}^{\text{miss}}$ (or $\boldsymbol{X}^{\text{miss}}_0$) after T-step noise-added part as $\boldsymbol{X}_T$ and the unnoised part of the data $\boldsymbol{X}^{\text{cond}}_0$ as masked trajectory information $\widetilde{\boldsymbol{X}}$.

Definition 2. Grid. In general, a spatial region is partitioned into $N \times L$ cells and each cell is regarded as a grid. In this paper, we will grid the trajectory data according to the spatial region and the adaptive resolution of each trajectory $\boldsymbol{X}^i$. Each trajectory data point only belongs to a certain grid, and a grid can contain multiple trajectory data points. In this way, we can generate an accompanying information matrix $\textbf{Grid} \in \mathbb{N}^{N\times L}$ for the trajectory data $\boldsymbol{X}$. At the same time, we store the total number of grids for all trajectories as $\mathcal{N} = \{\mathcal{N}^1, \mathcal{N}^2, \ldots, \mathcal{N}^i, \ldots, \mathcal{N}^N\}$, where the grid values for each point satisfy $0 \le \textbf{Grid}^{i,j} \le \mathcal{N}^i$.

Definition 3. Missing record matrix. Mask is the missing record matrix, $\textbf{Mask} \in [0,1]^{N\times L}$. We use the missing record matrix to defined $\boldsymbol{X}^{\text{cond}}$ and $\boldsymbol{X}^{\text{miss}}$, as the subsets of $\boldsymbol{X}$ corresponding to observed and missing data, respectively. Additionally, Taking into account the practical issue of random missing trajectory information, we uniformly

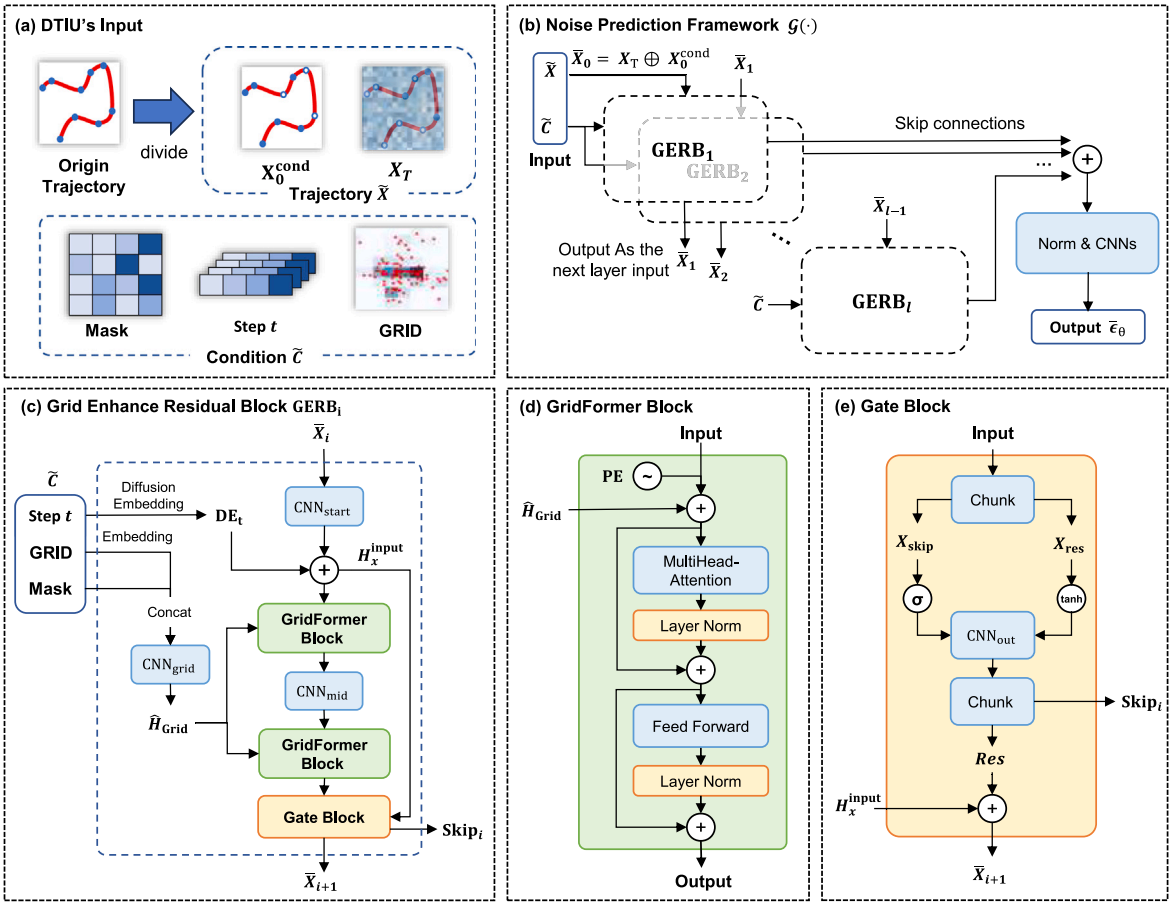


Fig. 2. The architecture of DTIU. **(a)** shows the input of DTIU, including trajectory input $\widetilde{X} = (X_T, X_0^{\text{cond}})$ and conditional input $\widetilde{C} = (\text{Grid}, \text{Mask}, \text{t})$. **(b)** shows the noise prediction framework $\mathcal{G}(\cdot)$ of DTIU. Trajectory input $\widetilde{X}$ is concat-operated to obtain $\bar{X}_0 = X_T \oplus X_0^{\text{cond}}$ for the first $GERB$ block, and conditional input $\widetilde{C}$ is processed according to (c). Through this structure, DTIU can obtain the predicted noise $\bar{\epsilon}_\theta$ to fit the noise ϵ we set. **(c)** shows the Grid Enhance Residual Block $GERB$, under the stacking of multiple $GERB$s, the trajectory data and grid information are fused and extracted to explore the temporal and spatial position relationship contained therein. **(d)** shows the GridFormer layer, which fully integrates the grid information $\hat{H}_{\text{Grid}}$ from (c). **(e)** is the gate control module, which divides the trajectory into blocks and fuses them to prevent the trajectory gradient from disappearing..

randomly sample a proportion α of points from the all-ones matrix $\mathbf{1} \in \mathbb{R}^{N\times L}$ and assign a value of 0 to the corresponding positions to obtain the missing record matrix $\mathbf{Mask} \in [0,1]^{N\times L}$. Then, we use **Grid**, **Mask** and diffusion step t as condition input $\widetilde{C}$, thus taking $\{\widetilde{X}, \widetilde{C}\}$ as model total input.

***Problem Definition* 1.**

While given trajectory $\boldsymbol{X}$, we define the problem of trajectory sequence imputation in unconstrained environments as training a network model p_θ to approximate the true distribution $q(\boldsymbol{X})$:

$$\max_\theta p_\theta(\boldsymbol{X}^{\text{miss}} | \boldsymbol{X}^{\text{cond}}) \text{ where } (\boldsymbol{X}^{\text{miss}}, \boldsymbol{X}^{\text{cond}}) \sim q(\boldsymbol{X}) \tag{1}$$

where p_θ is a probabilistic imputation model aimed at approaching the true conditional distribution $q(\boldsymbol{X}^{miss}|\boldsymbol{X}^{cond})$.[14]

4. Method

Fig. 2(a) and Fig. 2(b) illustrates the input and framework of the proposed DTIU model, which is designed for trajectory sequence imputation and consists of several key steps: data preprocessing, forward noising process, reverse denoising process (imputation), self-supervised training.

In the data preprocessing step, We first perform a Data Deformation on the trajectory data (Data Deformation), then we use masks to miss the trajectory data and perform data cutting to obtain the same length of data. We also make settings for different levels of noise addition. The detailed explanation of these steps can be found in Section 4.1.

In the forward noising process, the masked trajectory grid data as well as the conditional masks are encoded with the trajectory grid data at each step t into the embedding layer section. The above is then subjected to a concat operation with step embedding to compute the conditional embedding based on each step t and used as the conditional input to the diffusion model. Next, the processed trajectory data is input into the Diffusion Decoder. The Diffusion Decoder includes components such as the GridFormer, as shown in Fig. 2(d), 1-d convolutional layers, and the Fusion layer. To effectively reconstruct the noise and impute the missing trajectory sequences, the noise-adding process is iteratively executed to ensure the data converges to a Gaussian distribution and get final predicted noise. The detailed explanation of these steps can be found in Section 4.2.

And finally, we perform a specific training implementation of DTIU through a unique self-supervised training method, we divided the overall data into conditional data and data to be filled in, based on the assumptions in Chapter 3, and added grid-enhanced data, etc. as conditions along with input. The detailed explanation of these steps can be found in Section 4.3.1.

4.1. Data preprocessing

We used three diverse methods of data processing, as follow.

- **Data Deformation.** We employed the Min–Max scaling technique to constrain the data range, as we discerned that trajectory data are particularly sensitive to precision. The use of common nonlinear transformations significantly affects the outcomes, as minor decimal variances in latitude and longitude during transformation can lead to substantial positional discrepancies upon data restoration, thereby considerably impacting the numerical results. Fine-tuning based on the Min–Max transformation allowed us to concentrate on the precision of the decimal points, effectively resolving this issue.
- **Data Missing.** We first adopt random point omission to simulate the real-world challenge of incomplete data. Specifically, we generating random mask by randomly select 10% trajectory points location as missing points. Additionally, we considered the random block omission. This approach creates block masks for contiguous sections of the data sequence following the Markov chain principles, involves generating block mask by simulating a process similar to a geometric distribution. And thereby determining the average length of masked sequences. Specifically, the transition between states (from "masked" to "unmasked") depends on predefined parameters and the masking ratio. The main relationships are as follows:

$$p_m = \frac{1}{\mathrm{lm}}, \quad p_u = p_m \times \frac{\mathrm{mr}}{1-\mathrm{mr}}. \tag{2}$$

  where p_m is the probability of the end of the masking sequence and p_u is the probability of the end of the unmasked sequence. lm is the average length, mr is the masking ratio .
- **Data Cutting.** After data cleaning and preprocessing, each original trajectory has a huge difference in the number of trajectory points, which cannot evenly reflect the trajectory pattern information of the dataset. To normalize data, we confined each trajectory to a maximum length of $l_{\max}$. Trajectories exceeding $l_{\max}$ were trimmed to $l_{\max}$ points, while trajectories shorter than $l_{\max}$ were padded with zeros and their positions marked using a padding matrix **Mask**. Trajectories with fewer than $l_{\min}$ points were discarded. In this study, we set $l_{\max} = 200$ and $l_{\min} = 50$.
- **Adding Noise.** Determining the appropriate level of Gaussian noise, β, to introduce into the data has always been an intriguing area of exploration. We establish four different schedules during our experiments: cosine, sigmoid, quadratic, and linear. In our experiments, we opted for a quadratic schedule to set β unevenly, achieving favorable results. We establish an initial noise level $\beta_{\text{start}} = 0.0001$ and increased it to a final level of $\beta_{\text{end}} = 0.5$ step by step.

4.2. DTIU model

We have thoroughly examined numerous diffusion imputation architectures, noting that current generative spatio-temporal data imputation models predominantly rely on VAEs or GANs. However, there has been no prior work directly applying conditional diffusion models to spatio-temporal data imputation. Considering the similarities between spatio-temporal data and time series data, we introduce time series imputation models to spatio-temporal data imputation. Previous research has made some progress on this issue, such as the aforementioned CSDI [1], SSSDS4 [15], or MIDM [14], which are all based on the DiffWave framework. Therefore, we attempt to adapt these time series imputation models from the DiffWave architecture as a baseline for trajectory sequence completion and propose a more suitable completion model for general trajectory data. The proposed model framework, as depicted in Fig. 2, involves multiple stages, including the GridFormer, as depicted in Fig. 2(d), layers and one-dimensional convolutional layers, along with processed trajectory data. The detailed processes and mechanisms will be further introduced in the subsequent sections.

4.2.1. Imputation diffusion model

Denoising diffusion probabilistic models have demonstrated state-of-the-art performance for multivariate time series imputation.[1] Based on the premise that the diffusion process is a Markov process, the diffusion model aims to use the learned distribution $p_\theta(\boldsymbol{X})$ to approximate the true distribution $q(\boldsymbol{X})$, learning to map from latent space to original signal space by removing noise in the reverse process that is added sequentially in a Markovian manner during the so-called forward process.[15]

Step 1, The forward noising process can be parameterized as:

$$q(\boldsymbol{X}_t|\boldsymbol{X}_{t-1}) = \mathcal{N}(\boldsymbol{X}_t; \sqrt{1-\beta_t}\boldsymbol{X}_{t-1}, \beta_t I) \tag{3}$$

where β_t represents the noise level, that is, given x_{t-1}, then $X_T = \sqrt{1-\beta_t}x_{t-1}$. Given the original sequence x_0, we can ultimately obtain the noised data X_T. We replace α_t with $1-\beta_t$, yielding:

$$X_T = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon \tag{4}$$

where $\epsilon \sim \mathcal{N}(0, I)$ and $\bar{\alpha}_t = \prod_{i=1}^{t}\alpha_i$.

Step 2, The reverse denoising process can be parameterized as:

$$\begin{aligned} &p(\boldsymbol{X}_{t-1}|\boldsymbol{X}_t, \boldsymbol{X}_0) = \mathcal{N}(\boldsymbol{X}_{t-1}; \mu(\boldsymbol{X}_{t-1}|\boldsymbol{X}_t, \boldsymbol{X}_0), \sigma(t)), \\ &s.t.\ \mu(\boldsymbol{X}_{t-1}|\boldsymbol{X}_t, \boldsymbol{X}_0) = \frac{1}{\sqrt{\alpha_t}}\boldsymbol{X}_t - \frac{1-\alpha_t}{\sqrt{\alpha_t(1-\bar{\alpha}t)}}\epsilon \end{aligned} \tag{5}$$

where $\epsilon \sim \mathcal{N}(0, I)$, and the variance $\sigma(t)$ is solely dependent on t, specified as:

$$\sigma(t) = \frac{1-\bar{\alpha}t-1}{1-\bar{\alpha}t}(1-\alpha_t). \tag{6}$$

At this point, given x_0 and X_T, the network model outputs ϵ_θ, denoising X_T into x_{t-1}, that is:

$$\boldsymbol{X}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\boldsymbol{X}_t - \frac{1-\alpha_t}{\sqrt{\alpha_t(1-\bar{\alpha}t)}}\epsilon_\theta(\boldsymbol{X}_t, t) \tag{7}$$

To minimize the KL divergence between $q_\theta(\boldsymbol{X}_{t-1}|\boldsymbol{X}_t)$ and the true distribution $q(\boldsymbol{X}_{t-1}|\boldsymbol{X}_t)$, [1] demonstrated that the reverse process can be trained by solving the following optimization problem:

$$\min_\theta E_{\boldsymbol{X}_0\sim q(x_0),\epsilon\sim\mathcal{N}(0,I),t}\|(\epsilon - \epsilon_\theta(X_T, t))\|_2^2 \tag{8}$$

4.2.2. Noise prediction framework

As we know from the previous section, one of the keys to the completion task is to design a suitable noise prediction module $\epsilon_\theta(\cdot)$ to fit the noise, so as to recover the trajectory. In order to better obtain the predicted noise ϵ_θ, we, inspired by diffwave and CSDI, redesigned the noise prediction framework $\mathcal{G}(\cdot)$. This framework is the core part of DTIU, mainly including some projection functions and multi-layer Grid Enhance Residual Block GERB(·) as shown in Fig. 2(c).

In general, we input the masked trajectory information $\widetilde{X}$ and conditional information $\widetilde{C}$ into $\mathcal{G}(\cdot)$, use the built-in spatiotemporal encoder of GERB(·) to map the latent embeddings into the original data space, and superimpose the results of multiple layers of GERB(·). Finally, we use the projection function based on multiple convolution operations CNNs(·) to fit and predict the noise ϵ_θ, which can be generally expressed as:

$$\begin{aligned} &\epsilon_\theta = \mathcal{G}(\widetilde{X}, \widetilde{C}), \\ &\mathcal{G}(\widetilde{X}, \widetilde{C}) = \mathrm{CNNs}\left(\frac{1}{\sqrt{l}}\sum_{i=1}^{l}(\mathrm{GERB_i}(\widetilde{X},\ \widetilde{C}))\right) \end{aligned} \tag{9}$$

where the total trajectory information $\widetilde{X}$ includes the masked trajectory noise X_T after t-step noise addition, the observed trajectory X_0^{cond}, and the conditional information $\widetilde{C}$ includes condition mask **Mask**, Grid information **Grid**, and time steps t.

Grid Enhance Residual Block. By constructing multiple layers of Grid Enhance Residual Block GERB(·) and performing residual connections, the Grid information $H_{\mathbf{Grid}}$ can be fully integrated into the

trajectory data $\widetilde{X}$. In order to better predict the fitted noise ϵ_θ, we combine the gating module Gate(·), as shown in Fig. 2(e), with the carefully designed GridFormer(·), as shown in Fig. 2(d). And we use an auto-encoder to store the potential states of all noisy trajectories in chronological order.

Specifically, the process of GERB(·) can be divided into the following parts. Firstly, the input trajectory data $\widetilde{X}$ is decomposed into the noisy trajectory X_T and the conditional trajectory X_0^{cond}, which are then concatenated via the operation $\oplus$ to generate $\bar{X}_0$. Simultaneously, the conditional information $\widetilde{C}$ is decomposed into Grid information H_{Grid}, a mask Mask, and a position embedding PE, which are also concatenated to form $\bar{C}$.

$$\begin{aligned}\widetilde{\boldsymbol{X}} &= (\boldsymbol{X}_T, \boldsymbol{X}_0^{\text{cond}})\\ \widetilde{\boldsymbol{C}} &= (\textbf{Grid}, \textbf{Mask}, t)\\ \bar{\boldsymbol{X}}_0 &= \boldsymbol{X}_T \oplus \boldsymbol{X}_0^{\text{cond}}\end{aligned} \tag{10}$$

Then, the process of GERB(·) can be shown as:

$$GERB_i(\cdot) = \text{Gate}(\text{GridFormers}(\bar{\text{X}}_\text{i}, \widetilde{\boldsymbol{C}})) \tag{11}$$

Subsequently, a carefully designed trajectory encoder GridFormers(·) processes $\bar{X}$ and $\bar{C}$ to generate intermediate features $H_{\text{GridFormer}}$. This step aims to capture the spatial and temporal features of the trajectory data.

$$H_{\text{GridFormer}} = \text{GridFormers}(\bar{\text{X}}_\text{i}, \widetilde{\boldsymbol{C}}, t) \tag{12}$$

The $H_{\text{GridFormer}}$ is then split into skip connections X_{skip} and residual connections X_{res}. Through a gating mechanism Gate(·), weights W_{gate} are applied along with activation functions σ and tanh to process X_{skip} and X_{res}, controlling the flow of information and enhancing the expressive power of the features.

$$\text{X}_{\text{skip}}, \text{X}_{\text{res}} = \text{Chunk}(\text{H}_{\text{GridFormer}}) \tag{13}$$

$$Gate(\cdot) = W_{\text{gate}} * \sigma(X_{\text{skip}}) * \tanh(X_{\text{res}}) \tag{14}$$

GridFormer Block. The GridFormer Block is a critical component within the noise prediction framework $\mathcal{G}(\cdot)$, specifically designed to handle and enhance the spatiotemporal information in trajectory data. This module effectively extracts and fuses trajectory data with Grid information by integrating a temporal encoder and Grid embedding capabilities, thereby improving the accuracy of noise prediction. The processing flow of the GridFormer can be broken down into several key steps:

Initially, the input trajectory data $\bar{X}$ is passed through an upsampling operation $\text{CNN}_{\text{start}}$ and a fully connected layer FC, combined with the output of the diffusion decoder $DE(t)$. The ReLU activation function is applied to generate intermediate features H_X^{input}. Subsequently, the conditional information $\bar{C}_i$ is embedded via the operation $Emb(\cdot)$ to map the conditional information into a high-dimensional space. This is followed by upsampling through a one-dimensional convolutional network CNN_{grid} to obtain the fused embedding of Grid and time, denoted as $\hat{H}_{grid}$.

$$H_X^{\text{input}} = \text{ReLU}(\text{CNN}_{\text{start}}(\bar{\text{X}}_\text{i} + \text{FC}(\text{DE(t)}))) \tag{15}$$

$$\hat{H}_{\text{Grid}} = \text{CNN}_{\text{Grid}}(\text{Emb}(\textbf{GRID}) \oplus \textbf{Mask}) \tag{16}$$

In the second step, H_X^{input} and $\hat{H}_{\text{grid}}$ are fed into an intermediate one-dimensional convolutional network CNN_{mid} to further extract features and generate intermediate features H_X^{mid}. Subsequently, H_X^{mid} and $\hat{H}_{\text{Grid}}$ are input into the GridFormer to obtain the final feature representation $H_{\text{GridFormer}}$.

$$H_X^{\text{mid}} = \text{CNN}_{\text{mid}}(\text{GridFormer}(H_X^{\text{input}}, \hat{H}_{\text{Grid}})) \tag{17}$$

$$H_{\text{GridFormer}} = \text{GridFormer}(H_X^{\text{mid}}, \hat{H}_{\text{Grid}}) \tag{18}$$

Specifically, the process of GridFormer(·) can be divided into the following parts. Within the GridFormer, layer normalization LN and multi-head attention mechanism MultiHeadAttention(·) are used to process the input features X, generating attention features X_{attn}. Then, X_{attn} is processed by a fully connected layer FCs, combined with $\hat{H}_{\text{Grid}}$, and finally outputs the features X_{out} after layer normalization LN.

$$\begin{aligned}X_{\text{attn}} &= \text{LN}(X + \text{MultiHeadAttention}(X))\\ X_{\text{out}} &= \text{LN}(X + \text{FCs}(X_{\text{attn}})) + \hat{H}_{\text{Grid}}\end{aligned} \tag{19}$$

Here, the MultiHeadAttention(·) network consists of n_l layers of multi-head attention blocks and fully connected (FC) network blocks. For the ith attention head, the input latent state H^j is transformed as shown in the following formula, where the projection matrices W_i^Q, W_i^K, and W_i^V are learnable parameters, and LN is the layer normalization network.

$$\begin{aligned}&\text{MultiHeadAttn}(\cdot) = \text{Concat}(\text{head}_1, \ldots, \text{head}_{\text{n}_\text{l}}) * \text{W}\\ &\text{head}_\text{i} = \text{Attention}(X\text{W}_\text{i}^\text{Q}, X\text{W}_\text{i}^\text{K}, X\text{W}_\text{i}^\text{V})\\ &\text{Attention}(\text{Q}, \text{K}, \text{V}) = \text{softmax}(\frac{\text{Q} \cdot \text{K}^\text{T}}{\sqrt{d_k}}) * \text{V}\end{aligned} \tag{20}$$

Through this series of meticulous processing steps, the GridFormer module effectively captures and fuses trajectory data with Grid information, providing robust feature support for subsequent noise prediction.

4.2.3. Adaptive trajectory grid

In this section, we propose a data augmentation method for gridding trajectory data, aimed at addressing information loss. Building on this, we also propose a Adaptive Trajectory Grid, whose resolution is data-driven. Trajectories within adjacent grids often share similar spatial characteristics, which allows this method to not only optimize the expressiveness of trajectory location information but also helps the model learn these spatial features more efficiently.

This method typically involves dividing the area into multiple grids of fixed resolution and mapping all trajectory points to the corresponding cells, as illustrated in Fig. 3(a). This approach treats trajectory points located in the same cell as a unified district, assigning the same identifier to these points and simplifying the spatial information of the trajectory points.

As we assume that the trajectory point set of the entire data set is $D = \{(x_i, y_i)\}_{i=1}^N$, we can know that the difference of the data boundary is

$$\Delta_x = \max_i x_i - \min_i x_i, \qquad \Delta_y = \max_i y_i - \min_i y_i \tag{21}$$

Given the global bounding box of all trajectories, let Δ_x and Δ_y be its longitudinal and latitudinal spans. In implementation process, we select the smaller spans as source reference of ticks. And then select y_{ticks} or x_{ticks}, where y_{ticks} and x_{ticks} are set by the internal automatic tick locator of matplotlib.

Finaly, resolution is

$$r_{\text{dataset}} = \frac{1}{k}\min\{\Delta \text{y}_{\text{ticks}}, \Delta \text{x}_{\text{ticks}}\} \tag{22}$$

where $\Delta y_{\text{ticks}}, \Delta x_{\text{ticks}}$ are ticks spans. So that the trajectory area is partitioned into several segments, balancing spatial fidelity with computational cost. Each grid cell is assigned a unique token, which is embedded into a d-dimensional vector before entering the *GridFormer* block.

Compared with a fixed grid, Eq. (22) guarantees **(i)** scale-invariance across datasets with different map sizes, and **(ii)** a controllable upper bound on the number of cells And in experiment, we use k=8 which leads to predictable memory footprint, ensure the adaptability and consistency of the method in different datasets and different data.

This grid-based processing is particularly useful in general scenarios lacking external auxiliary information, such as road networks or real-time images, because obtaining precise geographic location information may not be feasible.

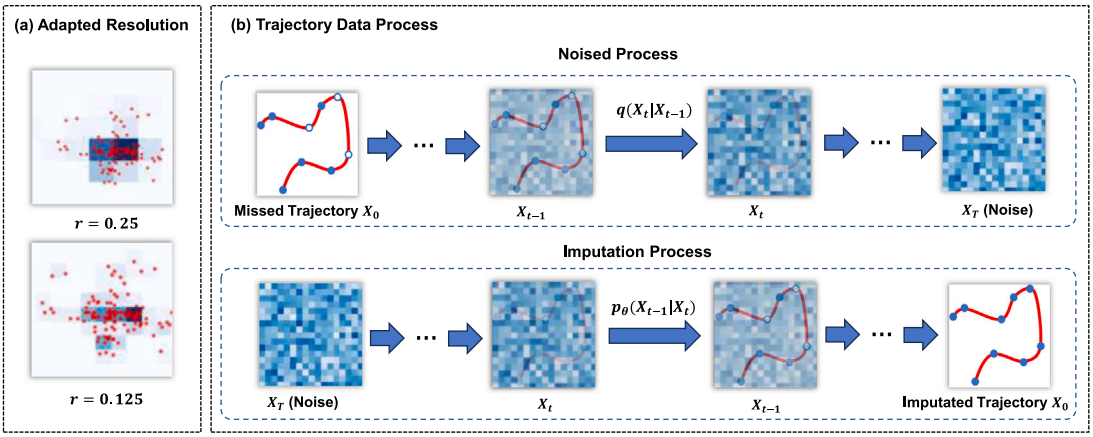


Fig. 3. DTIU's data process. (a) shows the gridding of trajectories at different resolutions. During training, we use the Adapted Trajectory Grid method, which will automatically select the appropriate resolution and correspond each point of the trajectory to a grid of appropriate size. (b) shows the trajectory data imputation process. Noise addition mainly exists in the training process. We will use a variety of methods to obtain different degrees of β_t and gradually add noise to the trajectory from X_0 to X_T. At this time, the data of X_T can be regarded as random noise. The imputation process can be regarded as the inverse process of noisy addition. We will obtain the predicted noise $\bar{\epsilon}_\theta$ from the trained model and denoise the sampled random noise based on the $\bar{\epsilon}_\theta$ to obtain the imputation data X_0.

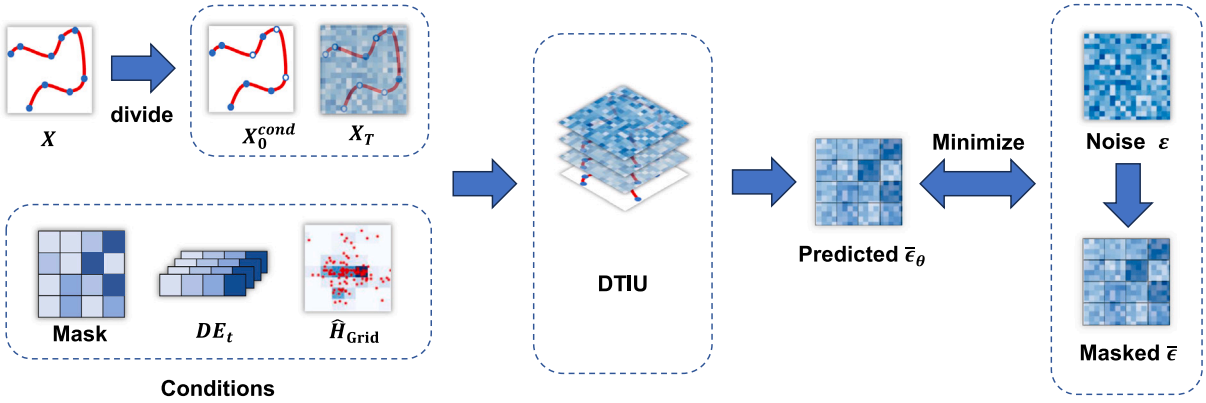


Fig. 4. Training process of self-supervised masking. Inspired by the masked language model, the model divides the raw data X into conditional trajectories X_0^{cond} and imputation targets X_0^{miss}. In real imputation, the imputation target is the trajectory with missing information, and $X_T{}^{\text{miss}}$ is obtained by adding random Gaussian noise, which we abbreviate as X_T. After inputting the model together with the conditional information, containing **Mask**, $\boldsymbol{DE}_t$ and $\hat{H}_{\text{Grid}}$., the training is performed by comparing the difference between the predicted noise and the random Gaussian noise.

4.3. Training and imputation of DTIU

Fig. 3(b) illustrates the process of trajectory data imputation using DTIU. We begin the imputation from random noise on the right side of the figure and gradually transform the noise into a plausible trajectory series through the reverse process of the conditional diffusion model. At each step t, the reverse process removes noise from the output of the previous step $p_\theta(x_{t-1}|x_t)$. The denoising process can take the trajectory sequence of the unmissed data (shown in the top left corner of the figure) as conditional input, allowing the model to utilize information observed for denoising.

4.3.1. Self-supervised training

We employ a training concept similar to the masked language modeling used in natural language processing, using **Mask** for artificially missing record matrices, where $\textbf{Mask} \in [0,1]^{N\times L}$. Through the missing record matrix, we divide into conditional data $\boldsymbol{X}_0^{\text{cond}}$ and imputation data $\boldsymbol{X}_0^{\text{miss}}$ according to the assumptions in Section 3, where $\boldsymbol{X}_0^{\text{cond}} = \boldsymbol{X}^{i,t}|\textbf{Mask}^{i,t} = 1$ and $\boldsymbol{X}_0^{\text{miss}} = \boldsymbol{X}^{i,t}|\textbf{Mask}^{i,t} = 0$. During training, we can follow the training process of the unconditional model in Section 4.2.1, adding $\boldsymbol{X}_0^{\text{cond}}$ and grid data **Grid** from the map, which makes the optimization target become:

$$\min_\theta E_{\boldsymbol{X}_0\sim q(x_0),\epsilon\sim N(0,1),t}\|(\epsilon-\epsilon_\theta(\widetilde{\boldsymbol{X}},\widetilde{\boldsymbol{C}}))\|_2^2 \tag{23}$$

In this context, ϵ should be consistent with the dimension of the input $\boldsymbol{X}_0^{\text{cond}}$. During training, we treat the fabricated $\boldsymbol{X}_0^{\text{miss}}$ as missing values of the trajectory for training, which are set to 0 or NaN. The specific algorithm can be seen in Algorithm 1, and the training process based on mask language modeling concept is shown in Fig. 4.

Algorithm 1 Training of DTIU.

Input: Distribution of training data $q(x_0)$, number of iterations $\mathcal{N}_{\text{iter}}$, sequence of noise levels $\{\alpha_t\}$, and integrated trajectory data $\{\widetilde{\boldsymbol{X}},\widetilde{\boldsymbol{C}}\}$

Output: Trained denoising function ϵ_θ

1: **for** $i = 1$ to $\mathcal{N}_{\text{iter}}$ **do**
2: $t \sim Uniform(\mathbf{1},...,T), x_0 \sim q(x_0)$
3: $\boldsymbol{X}_0^{\text{cond}} = \boldsymbol{X}\odot\textbf{Mask}, \boldsymbol{X}_0^{\text{miss}} = \boldsymbol{X}\odot(1-\textbf{Mask})$, $\epsilon\sim N(0,I)$, where ϵ and $\boldsymbol{X}$ have the same dimensions
4: Add noise to $\boldsymbol{X}_0^{\text{miss}}$ to get $x_T = \sqrt{\bar{\alpha}_T}x_0^{\text{miss}} + \sqrt{\mathbf{1}-\bar{\alpha}_T}\epsilon$
5: Perform a gradient step according to Eq. (23) :$\nabla_\theta\|(\epsilon-\epsilon_\theta(\widetilde{\boldsymbol{X}},\widetilde{\boldsymbol{C}}))\|_2^2$
6: **end for**

4.3.2. DTIU imputation

During the imputation inference, we initially sample random noise ϵ from a standard Gaussian, denoted as X_T, and through the reverse process of the conditional diffusion model, at each time step t, we

Table 1
Statistics of the 4 datasets after preprocessing.

Dataset	Location	Scenarios	Duration	Traj pair	Node num
Geolife	Beijing	Urban Traffic	5 years	993	105 230
EastSeaAIS	East Sea	Open ocean	12 months	192	19 598
DCIC2021	Taiwan Strait	Open ocean	11 months	418	80 242
LocaRDS	Open Sky	Airspace	3600 s	921	180 485

Algorithm 2 Imputation of DTIU.

Input: Distribution of training data $q(x_0)$, number of iterations $\mathcal{N}_{iter}$, sequence of noise levels $\{\alpha_t\}$, observed trajectory data $\{\boldsymbol{X}_0^{cond}, \widetilde{\boldsymbol{C}}\}$,Trained denoising function ϵ_θ, and number of sampling n_{sample}

Output: Imputation data $\boldsymbol{X}_0^{\text{miss}}$

1: Regard random noise $\epsilon \sim N(\mathbf{0}, I)$ as $\boldsymbol{X}_T$, where ϵ have the same dimensions with $\boldsymbol{X}$
2: **for** $i = T$ to 1 **do**
3: $t \sim Uniform(\mathbf{1}, ..., T), x_0 \sim q(x_{\mathbf{0}})$
4: $\boldsymbol{X}_T^{\text{miss}} = \epsilon \odot (1 - \mathbf{Mask})$
5: denoise $\boldsymbol{X}_T^{\text{miss}}$ to get $\boldsymbol{x}_{t-\mathbf{1}}^{\text{miss}} = \frac{\mathbf{1}}{\sqrt{\alpha_t}}\boldsymbol{x}_t - \frac{\mathbf{1}-\alpha_t}{\sqrt{\alpha_t(\mathbf{1}-\bar{\alpha}_t)}}\epsilon_\theta(\boldsymbol{x}_t, t)$
6: generating multiple Imputation(Sampling) with hyperparameter n_{sample}
7: **end for**

utilize the $\epsilon_\theta(\boldsymbol{X}_0^{\text{cond}}, \widetilde{C})$ generated by the trained network using the given missing data x_0^{cond}. This allows us to obtain $\boldsymbol{X}_{t-1}^{\text{miss}}$, progressively denoising to ultimately produce the generated imputation data $\boldsymbol{X}_0^{\text{miss}}$, gradually transforming noise into a plausible completed trajectory sequence. The specific algorithm is as above Algorithm 2.

5. Experiment

5.1. Experiment setup

In this section, we demonstrate the efficacy of DTIU for the imputation of trajectory sequences. Four datasets were employed, and our methodology is explicated in three facets concerning the construction of the datasets and evaluation metrics.

5.1.1. Dataset

We evaluated the performance of the DTIU on 4 trajectory datasets: Geolife, EastSeaAIS, DCIC2021, and LocaRDS.

- **Geolife**.[1] The Geolife dataset is collected from April 2007 to August 2012 by 182 users for a geospatial project at Microsoft Research Asia, and represents urban scenario. The GPS trajectories in the dataset consist of sequences of timestamped points, including information on latitude, longitude, and altitude. Such datasets are extensively used for tasks such as trajectory prediction and travel time estimation.
- **EastSeaAIS**.[2] The EastSeaAIS dataset comprises trajectory data transmitted by AIS equipment on ships in the East China Sea from July 2018 to June 2019. As a maritime scenario, it originates from real historical trajectories of ships at sea and encompasses multi-dimensional information. Datasets of this kind are widely used in applications like assessing the navigational state of ships and detecting collision probabilities.
- **DCIC2021**.[3] The DCIC2021 dataset, from the Digital China Innovation Contest 2021, provides simulated human drift coordinates and marine environmental factors from December 2019 to October 2020. This dataset is used in maritime scenarios, with the goal of building drift models to accurately predict the drift trajectories of persons overboard, thereby minimizing the size of the search area and reducing search and rescue efforts.
- **LocaRDS**.[4] The LocaRDS dataset is an open reference dataset of real-world crowdsourced flight data from the OpenSky Network, containing more than 500 million measurements from over 50 million transmissions recorded by 323 sensors. This dataset is used for aerospace scenarios and can be employed for analysis in spatio-temporal tasks.

The entire datasets were partitioned into non-overlapping training, testing, and validation sets with a ratio of 8:1:1. Effective datasets were compiled by segmenting and filtering valid trajectories, with detailed data presented in the Table 1 below.

5.1.2. Baseline

In scenarios where information loss is prevalent and conventional models reliant on road network data are inadequate for imputation, we benchmark our proposed DTIU against several widely-used models for multidimensional time series imputation, as well as state-of-the-art models. We consider both deterministic and generative models for comprehension thought.

Deterministic Models

- KNN [24]: Uses the average of the k-most similar variables for imputation; we set k=10 in our experiments.
- BRITS [27]: Utilizes recurrent dynamics to estimate missing values based RNN in multivariate time series.
- SAITS [31]: Learns the missing values from a weighted combination of two Diagonal Masked Self-Attention (DMSA) blocks.
- TimesNet [46]:Captures complex temporal periodicity by transforming 1-d data into 2-d tensors.
- iTransformer [47]: Applies attention mechanism and FFN on inverted dimensions and learns nonlinear representations for each token.
- TimeMixer++[48]: Employs multi-scale and multi-resolution pattern extraction to deliver SOTA performance.

Generative Models

- GP-VAE [41]: Deep probabilistic model for imputing multivariate time series that integrates concepts from VAE with Gaussian Processes (GP).
- SSGAN [40]: Viewed as a GAIN variant with bidirectional recurrent encoders and decoders.
- CSDI [1]: Employs a conditional diffusion model and uses the Diffwave architecture for estimating missing values in multivariate time series.
- SSSD^{S4}[15]: Adapts recent advances in state-space models as denoising modules, combining them with Diffwave to facilitate imputation.

[1] https://www.microsoft.com/en-us/download/details.aspx?id=52367
[2] https://osdds.nsoas.org.cn/DataRetrieval
[3] https://www.datafountain.cn/datasets/5968
[4] https://zenodo.org/records/4302265

Table 2
Performance comparison for deterministic and probabilistic models, the **bolded** ones are the best results for that metric, and the underlined ones are the second best.

Dataset		Geolife				LocaRDS				DCIC2021				EastSeaAIS			
Models type	Method	GridACC	MAE	RMSE	CRPS	GridACC	MAE	RMSE	CRPS	GridACC	MAE	RMSE	CRPS	GridACC	MAE	RMSE	CRPS
Deterministic	KNN-10	0.1533	5.1012	8.048	–	0.0177	0.1608	0.220	–	0.3394	5.4657	8.825	–	0.0493	3.2093	4.030	–
	BRTIS	0.6498	0.8859	1.789	–	0.2170	0.0226	0.0630	–	0.8916	0.3812	1.246	–	0.8910	0.3572	1.246	–
	TimesNet	0.3561	3.158	1.822	–	0.0625	0.0443	0.0881	–	0.7006	0.9452	2.035	–	0.4911	0.5581	0.9050	–
	SAITS	0.7186	0.9635	3.701	–	0.4343	0.0167	0.0901	–	0.8974	0.2190	0.3730	–	0.6928	0.3544	1.064	–
	iTransformer	0.3908	2.344	4.956	–	0.1719	0.0370	0.0913	–	0.8641	0.2411	0.6308	–	0.2214	1.024	1.521	–
	timemixer++	0.0635	5.613	9.517	-	0.0238	0.0457	0.0809	-	0.6743	1.5372	3.3418	-	0.2096	1.1624	1.914	-
Probabilistic	GP-VAE	0.0705	10.233	13.33	0.8603	0.0010	0.1489	0.176	0.1594	0.3395	3.9672	6.104	0.7995	0.0098	4.4873	5.986	0.8915
	SSSD	0.0677	4.1287	5.108	0.9601	0.0020	0.1698	0.213	0.1227	0.6814	1.7210	3.340	0.3289	0.0108	3.5766	3.695	0.6044
	USGAN	0.6425	0.9195	2.158	0.0743	0.2000	0.0232	0.060	0.0256	0.8941	0.2827	0.956	0.0522	0.5000	0.4938	0.925	0.1057
	CSDI	0.8776	0.2212	0.5947	0.0106	**0.9533**	**0.0064**	0.0024	0.0073	0.9581	0.1453	0.517	0.0218	0.7826	0.1745	**0.0121**	0.0331
	STIOS	0.9060	0.1801	0.4561	-	0.9475	0.0079	0.0020	-	–	–	–	-	–	–	–	–
DTIU		**0.9264**	**0.1398**	**0.3766**	**0.0080**	0.9497	0.0078	**0.0019**	**0.0068**	**0.9664**	**0.0714**	**0.4093**	**0.0115**	**0.8983**	**0.0954**	0.0158	**0.0103**

- STIOS [49]: Utilizes Diffwave architecture, as the preliminary model and predecessor to TIDU.

These models are evaluated under similar conditions to ensure a fair comparison. For each model, specific parameters are adjusted to optimize performance on the task of imputing missing trajectory sequences. The deterministic models tend to offer predictions based on learned patterns without randomness, whereas the generative models provide a probability distribution from which multiple plausible imputed values can be drawn. The choice between deterministic and generative models may depend on the specific application and whether uncertainty quantification of the imputed values is necessary. [50–52]

5.1.3. Evaluation metrics

To rigorously evaluate the performance of models in imputing missing data, we propose four metrics: Mean Absolute Error (MAE), Root Mean Square Error (RMSE), Grid Prediction Accuracy (Grid-ACC), and the Continuous Ranked Probability Score (CRPS). These metrics serve as quantitative measures of prediction accuracy, each capturing different aspects of the imputation quality. The formulas and their purposes are below:

- Root Mean Square Error (RMSE). RMSE measures the square root of the average squared differences between the predicted values ($\mathcal{X}^i$) and the true values ($\mathcal{Y}^i$). A lower RMSE indicates better predictive accuracy.
- Mean Absolute Error (MAE). MAE calculates the average of the absolute differences between predictions and actual observations. Like RMSE, a smaller MAE suggests better performance.
- Grid Prediction Accuracy (Grid-ACC). Grid-ACC is a domain-specific metric that measures the accuracy of predictions on a discretized grid (common in spatial data applications). It is the percentage of the number of correct grid cell predictions N_{acc} over the total number N_{grid}.
- Continuous Ranked Probability Score (CRPS)

$$\text{CRPS}(F, x) = \int_{-\infty}^{\infty} (F(y) - H(y - x))^2 dy \tag{24}$$

CRPS assesses the accuracy of probabilistic predictions. $F(y)$ is the predicted cumulative distribution function (CDF), x is the observed value, and $H(y-x)$ is the Heaviside step function, which equals 1 when $y \leq x$ and 0 otherwise. The CRPS is calculated as the integral of the squared difference between the predicted CDF and the step function representing the observation. It reflects the total inconsistency between the predicted distribution and the actual observation, with a lower CRPS indicating that the predicted distribution is closer to the actual distribution.

Each metric offers a unique perspective on imputation quality; RMSE and MAE focus on the magnitude of errors, Grid-ACC on the accuracy specific to spatial resolution, and CRPS on the quality of probabilistic forecasts. By employing all these metrics, we can obtain a comprehensive understanding of a model's performance across different dimensions of accuracy.

5.1.4. Other settings

All experiments were conducted on Ubuntu 18.04 equipped with Tesla V100S-PCIE 32 g GPU. We implement DTIU and all baselines based on PyTorch 1.13.0 and Python 3.7. We set the batch size to 64 and the initial learning rate to 0.001, and use the MultiStepLR method to adjust the learning rate. In the noise prediction module, we set the initial encoding embedding size d to 256, channel to 64, and the number of layers of GREB to 4. The attention head of GridFormer is 8, and the hidden dimension of the grid is set to 128. The mask length lm is 3 and the mask rate pm is 10%. The dropout rate is 0.1. Finally, we used the Adam optimizer for parameter optimization, with a weight decay set to 1e-6, and set the training epochs to 1000, with validation conducted every 5 epochs. Other baseline models have the same settings as DTIU, with 256 hidden dimensions, while other settings follow their default settings.

5.2. Results

This section will compare and analyze the results from four dimensions. First, we raise questions from 2 perspectives and show the overall performance of DTIU. Second, we care about the robustness and robustness. Finally, we focus on the problem of visual display of probabilistic imputation.

Table 2 shows the performance comparison of DTIU and selected baseline methods on four real datasets. Specifically, we compare the performance of the best models after training according to the four evaluation metrics described in the 5.1.3 section. We first compare these models with deterministic models, resulting in the observations in Table 2, where the best results are highlighted in bold and the second best results are highlighted in underline.

5.2.1. Are existing methods and DTIU capable of imputation tasks in unconstrained environment?

We can observe that in unconstrained environment scenarios, traditional methods such as KNN are not sufficient to complete the imputation task of spatiotemporal trajectory data. Obviously, sequence-based methods (TimesNet, SAITS and timemixer++) perform significantly better than simple traditional methods, that is, trajectory imputation requires the model to learn the correlation between spatiotemporal sequences of trajectories, and simple models alone may not be able to complete the task of trajectory imputation. However, the more advanced sequence-based methods still perform poorly and show unstable performance in different tasks. For example, the BRTIS algorithm performs well on land and maritime data, but has very poor accuracy on aerial data, which is significantly lower than the other 3 datasets. It is worth noting that when we repeated the experiment, the iTransformer model reached 90% accuracy in several trials and exceeded TIDU. However, the model had a large variance, as one of the test set accuracy was only 0.7127. At the same time, DTIU outperforms all non-generative baseline models and shows strong advantages in trajectory imputation and recovery. We further compare it with the generative probability model in Table 2 as follows. It can be seen that DTIU has achieved highly competitive performance in all indicators of the 3 tasks of the 4 real datasets, and maintains stable results on datasets of different sizes.

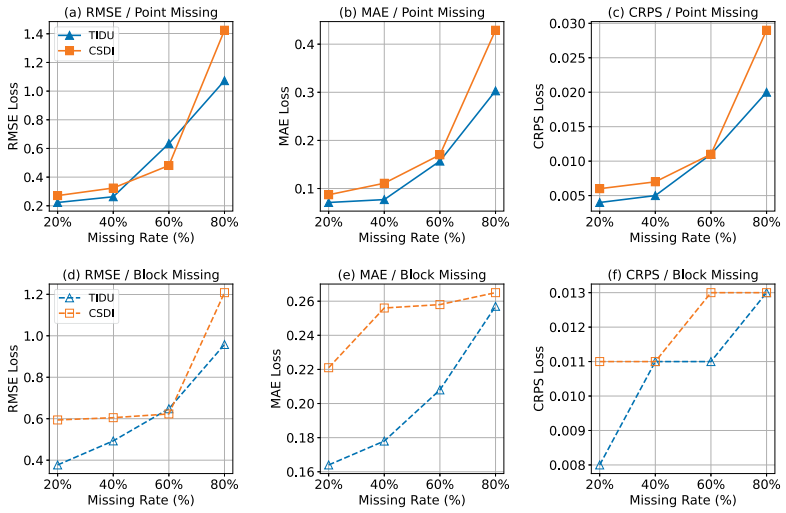


Fig. 5. Performance of DTIU under different missing rates and point missing & block missing conditions on GeoLife dataset. The first row of sub-figures is point missing, and the second row is block missing. Each sub-figure shows the interpolation performance of the CSDI model and our DTIU model from missing rate 20% to 80%.

5.2.2. *Why do most models perform poorly in GridACC?*

In Table 2, we note that, most time series interpolation models perform poorly in terms of trajectory grid prediction accuracy (GridACC). In fact, GridACC is an intuitive reflection of the completeness of trajectory data, representing the accuracy of the model in a coarse-grained manner. However, the sluggish performance of most models highlights the challenging nature of the problem, indicating that most models treat trajectories as ordinary time series and ignore the spatiotemporal information. In particular, the model SSSD that introduces a structured approach performs poorly on trajectory data. This may be due to the inability of structured state-space models to effectively learn trajectory data, while DTIU improves the accuracy and achieves stable results by making full use of the spatial information of the trajectory and the GridFormer layer.

5.2.3. *How will DTIU be affected under different data sparsity conditions?*

We conducted experiments to assess the robustness of data sparsity issues in the face of missing data. To simulate the effects of data sparsity, we increased the percentage of missing locations in the trajectories from 20% to 80% and employed two random masking strategies for masked training. We used random masking, which entails random point-wise missingness in the data, and random block masking (state Markov chain) to generate masks, indicating that missing data would occur in contiguous sequences (with the length of consecutive missingness controlled by a geometric distribution). We compared our model with the recently advanced model CSDI in Fig. 5.

It can be seen that when data is missing in large quantities due to external factors, it is more difficult to recover correctly; therefore, almost all indicators have a certain degree of decline. Nevertheless, the MAE of our model is about 0.05 higher than the baseline, and even in the extreme case of 80% missing data, it is 0.01 higher than the CSDI model. These results show that our proposed model maintains a certain robustness under different data sparsity conditions while retaining performance advantages. In addition, by comparing the results of random block masks and random point masks, random point missing is easier to recover than random block missing. We speculate that this is because the randomness of point missing is higher, the information loss is relatively dispersed, and the model is easier to infer and fill in with existing data. And in both types of missing, the error rate increases sharply after the data loss rate reaches 40%. Based on this, we believe that trajectory data is no longer suitable for recovery after the loss rate exceeds 40%.

Table 3
Ablation Study on EastSeaAIS dataset.

Method	GridACC	RMSE	MAE	CRPS
w/o GridFormer	0.7932	0.3811	0.1669	0.0324
w/o A. T. Grid	0.8734	0.1985	0.1493	0.0130
w/o Gate	0.8947	0.1475	0.0984	0.0129
DTIU	**0.8983**	**0.1396**	**0.0954**	**0.0103**

w/o GridFormer: We replace the double-layer Transformer layer in CSDI with the GridFormer layer. **w/o A. T. Grid**: We use a zero matrix of the same shape to replace the Adapted Trajectory Grid method and grid encoding information. **w/o Gate**: We only trunk result and use a single nonlinear function.

Table 4
Different resolutions on EastSeaAIS dataset.

Metrics/K	2	4	8	12
GridACC	0.9632	0.9515	0.8983	0.8984
MAE	0.1352	0.1266	0.0954	0.0962

5.2.4. *How do the GridFormer layer and the adapted trajectory grid method improve model performance?*

We added our designed GridFormer to $GREB$. In terms of basic structure, Transformer enables GridFormer to effectively process sequence data and capture long-distance dependencies. It is worth noting that there is no previous classic diffusion model for the trajectory field. We independently designed a special distribution for trajectories, combined it with the above-mentioned Adapted Trajectory Grid and converted it into an embedded vector through multiple embedding layers and convolution layers, that is, the implicit vector embedding $\hat{H}_{\text{Grid}}$. GridFormer uses the implicit vector embedding introduced by the grid layer to better integrate the trajectory position information into the feature. Although our method is the first to target unconstrained scenarios, we still verify the effectiveness of the GridFormer layer and Adapted Trajectory Grid method through ablation experiments on the EastSeaAIS dataset. We repeat the experiment ten times and list the results in Table 3. The comparison shows that our proposed method can obviously extract the spatiotemporal information of the trajectory more fully, which is also strongly reflected in the high accuracy of GridACC. With other hyperparameters unchanged, we selected different grid resolutions k to conduct experiments on the EastSeaAIS in Table 4. Considering both efficiency and economy, k=8 is the most balanced parameter.

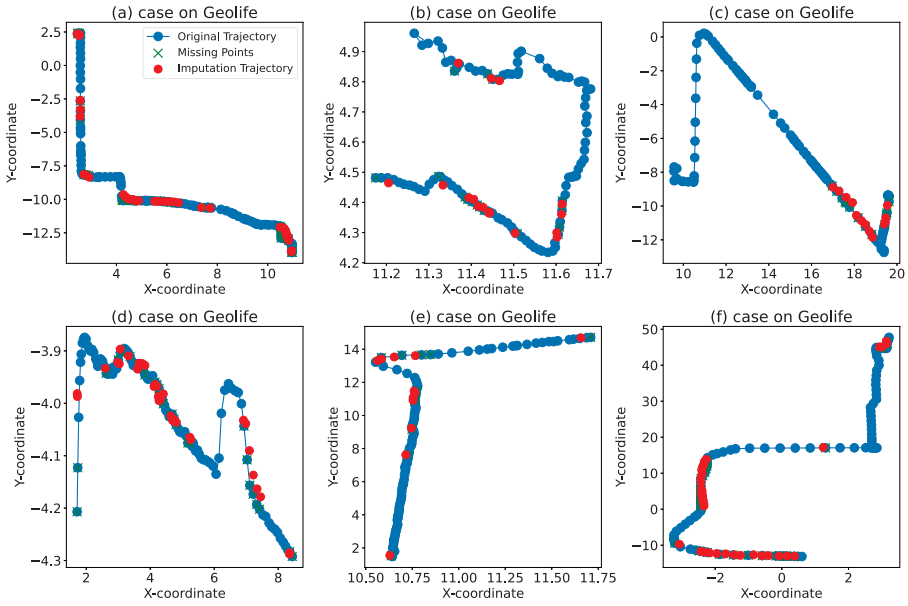


Fig. 6. Example of the trajectory imputation in Geolife dataset. Each sub-graph contains a trajectory of various categories such as walking trajectory, vehicle trajectory, and flight trajectory, as well as the imputation results. The horizontal and vertical axes represent the transformed coordinate values.

5.2.5. Case study and computational complexity

As mentioned earlier, one of the advantages of applying probabilistic models to multivariate imputation is that they can generate various plausible imputations, which is beneficial for uncertainty estimation. We provide a visual example of imputation by DTIU on the Geolife dataset in Fig. 6 . Blue circles represent the original trajectory values, red circles represent the imputed values, and green crosses signify the missing values. To generate the visualization of probabilistic imputation, we produce plausible imputations 100 times. As shown in the figure, our model fits the original time series well, and these results suggest that DTIU is capable of generating accurate estimations for missing values with high precision.

We report complexity with metrics measured on Section 5.1.4. Our model has 3.16 M parameters and 9.683 GFLOPs for all DTIU diffusion steps. And the theoretical complexity for one *ResidualBlockGrid* forward pass is summarized as $\mathcal{O}\left(B\,K\,L^2\,C\;+\;B\,K\,L\,(C^2+S\,C)\right)$. Nevertheless, Our peak memory is 0.06 GB and end-to-end inference of 200 time steps takes 18.16 ms/step, well within real-time requirements ($\lesssim 100$ ms).

6. Conclusion

In this work, we introduce a self-supervised method for trajectory imputation that is applicable across unconstrained scenarios. As the current state-of-the-art in generative modeling approaches, synergizes deterministic models with conditional diffusion models, enhancing positional information representation through grid-based encoding. This effectively addresses both spatio-temporal correlations and uncertainties. Testing on four real-world datasets demonstrates that DTIU surpasses existing advanced baseline models in various data sets under different information-missing scenarios, exhibiting particularly high competitiveness and superiority in open marine and urban land scenarios. Furthermore, this represents the first unified model to employ diffusion models in conjunction with representation learning for the task of trajectory imputation in universal scenarios, effectively mitigating the loss of spatio-temporal information in unconstrained environments. Notably, we showcase examples of qualitative improvements in imputation quality that are even visible to the naked eye and employ training methods akin to masked language modeling used in natural language processing field. We consider the proposed technique as a highly promising generative approach in the domain of spatio-temporal trajectory data. This opens up possibilities for the recovery of significant trajectories as well as generation, marking a step forward in the field.

CRediT authorship contribution statement

Zhijing Hu: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Hao Yan:** Writing – review & editing, Methodology, Conceptualization. **Kuihua Huang:** Writing – review & editing, Supervision, Funding acquisition. **Jincai Huang:** Writing – review & editing, Supervision, Funding acquisition. **Zhong Liu:** Writing – review & editing, Supervision, Funding acquisition. **Changjun Fan:** Writing – review & editing, Supervision, Resources, Funding acquisition.

Declaration of competing interest

This study was supported by the National Natural Science Foundation of China (NSFC, 72421002, 62206303), Science and Technology Innovation Program of Hunan Province (2023RC3009), Foundation Fund Program of National University of Defense Technology (JS24-05) and The Major Science and Technology Projects in Changsha, China (kq2301008). The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The data that has been used is confidential.

References

[1] Y. Tashiro, J. Song, Y. Song, S. Ermon, CSDI: Conditional Score-based Diffusion Models for Probabilistic Time Series Imputation, Cornell University, 2021, ArXiv.

[2] G. Zerveas, S. Jayaraman, D. Patel, A. Bhamidipaty, C. Eickhoff, A transformer-based framework for multivariate time series representation learning, in: Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining, 2021, pp. 2114–2124.

[3] T. Xia, Y. Qi, J. Feng, F. Xu, F. Sun, D. Guo, Y. Li, AttnMove: History enhanced trajectory recovery via attentional network, Proc. AAAI Conf. Artif. Intell. (2022) 4494–4502.

[4] B.L. Smith, M.J. Demetsky, Traffic flow forecasting: comparison of modeling approaches, J. Transp. Eng. 123 (4) (1997) 261–266.

[5] S. Shekhar, B.M. Williams, Adaptive seasonal time series models for forecasting short-term traffic flow, Transp. Res. Rec. 2024 (1) (2007) 116–125.

[6] M.I. Jordan, Serial order: A parallel distributed processing approach, in: Neural-Network Models of Cognition - Biobehavioral Foundations, Advances in Psychology, 1997, pp. 471–495.

[7] J.L. Elman, Finding structure in time, Cogn. Sci. (1990) 179–211.

[8] H. Sun, C. Yang, L. Deng, F. Zhou, F. Huang, K. Zheng, PeriodicMove: Shift-aware human mobility recovery with graph neural network, in: Proceedings of the 30th ACM International Conference on Information & Knowledge Management, 2021.

[9] Y. Huang, H. Bi, Z. Li, T. Mao, Z. Wang, STGAT: Modeling spatial-temporal interactions for human trajectory prediction, in: 2019 IEEE/CVF International Conference on Computer Vision, ICCV, 2019.

[10] H. Jeon, J. Choi, D. Kum, SCALE-net: Scalable vehicle trajectory prediction network under random number of interacting vehicles via edge-enhanced graph convolutional neural network, in: 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, 2020.

[11] J. Jiang, D. Pan, H. Ren, X. Jiang, C. Li, J. Wang, Self-supervised trajectory representation learning with temporal regularities and travel semantics, in: 2023 IEEE 39th International Conference on Data Engineering, ICDE, IEEE, 2023, pp. 843–855.

[12] M. Qi, J. Qin, Y. Wu, Y. Yang, Imitative non-autoregressive modeling for trajectory forecasting and imputation, in: 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR, 2020.

[13] H. Miao, J. Shen, J. Cao, J. Xia, S. Wang, MBA-STNet: Bayes-enhanced discriminative multi-task learning for flow prediction, IEEE Trans. Knowl. Data Eng. 35 (7) (2023) 7164–7177.

[14] X. Wang, H. Zhang, P. Wang, Y. Zhang, B. Wang, Z. Zhou, Y. Wang, An Observed Value Consistent Diffusion Model for Imputing Missing Values in Multivariate Time Series.

[15] J. Alcaraz, N. Strodthoff, Diffusion-based time series imputation and forecasting with structured state space models, 2022.

[16] M. Liang, B. Yang, R. Hu, Y. Chen, R. Liao, S. Feng, R. Urtasun, Learning lane graph representations for motion forecasting, in: Computer Vision – ECCV 2020, in: Lecture Notes in Computer Science, 2020, pp. 541–556.

[17] Y. Chen, X. Li, G. Cong, Z. Bao, C. Long, Y. Liu, A.K. Chandran, R. Ellison, Robust road network representation learning, in: Proceedings of the 30th ACM International Conference on Information & Knowledge Management, 2021.

[18] S.B. Yang, C. Guo, J. Hu, J. Tang, B. Yang, Unsupervised path representation learning with curriculum negative sampling, in: Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, 2021.

[19] Y. Liu, R. Yu, S. Zheng, E. Zhan, Y. Yue, NAOMI: Non-Autoregressive Multiresolution Sequence Imputation, Cornell University, 2019, ArXiv.

[20] B. Yang, G. Yan, P. Wang, C.-Y. Chan, X. Song, Y. Chen, A novel graph-based trajectory predictor with pseudo-oracle, IEEE Trans. Neural Netw. Learn. Syst. (2022) 7064–7078.

[21] O.D. Anderson, G.E.P. Box, G.M. Jenkins, Time series analysis: Forecasting and control, Stat. (1978) 265.

[22] E. Zivot, J. Wang, Vector autoregressive models for multivariate time series, in: Modeling Financial Time Series with S-Plus®, 2003, pp. 369–413.

[23] I.R. White, P. Royston, A.M. Wood, Multiple imputation using chained equations: Issues and guidance for practice, Stat. Med. (2011) 377–399.

[24] A.T. Hudak, N.L. Crookston, J.S. Evans, D.E. Hall, M.J. Falkowski, Nearest neighbor imputation of species-level, plot-scale forest structure attributes from LiDAR data, Remote Sens. Environ. (2008) 2232–2245.

[25] M. Berglund, T. Raiko, M. Honkala, L. Kärkkäinen, A. Vetek, J. Karhunen, Bidirectional recurrent neural networks as generative models, in: Neural Information Processing Systems, 2015.

[26] Z. Che, S. Purushotham, K. Cho, D. Sontag, Y. Liu, Recurrent neural networks for multivariate time series with missing values, Sci. Rep. (2018).

[27] W. Cao, D. Wang, J. Li, H. Zhou, L. Li, Y. Li, BRITS: Bidirectional recurrent imputation for time series, in: Neural Information Processing Systems, 2018.

[28] T.-Y. Choi, J. Kang, J.-H. Kim, RDIS: Random Drop Imputation with Self-Training for Incomplete Time Series Data, Cornell University, 2020, ArXiv.

[29] S. Shukla, B. Marlin, Multi-time attention networks for irregularly sampled time series, 2021, ArXiv: Learning.

[30] Q. Suo, W. Zhong, G. Xun, J. Sun, C. Chen, A. Zhang, GLIMA: Global and local time series imputation with multi-directional attention learning, in: 2020 IEEE International Conference on Big Data, Big Data, 2020.

[31] W. Du, D. Côté, Y. Liu, Saits: Self-attention-based imputation for time series, Expert Syst. Appl. 219 (2023) 119619.

[32] J. Morrill, C. Salvi, P. Kidger, J. Foster, Neural rough differential equations for long time series, in: International Conference on Machine Learning, 2021.

[33] Y. Rubanova, R.T. Chen, D. Duvenaud, Latent ordinary differential equations for irregularly-sampled time series, in: Neural Information Processing Systems, 2019.

[34] A. Cini, I. Marisca, C. Alippi, Filling the g_ap_s: Multivariate time series imputation by graph neural networks, 2021, arXiv preprint arXiv:2108.00298.

[35] Y. Chen, Z. Li, C. Yang, X. Wang, G. Long, G. Xu, Adaptive graph recurrent network for multivariate time series imputation, in: International Conference on Neural Information Processing, Springer, 2022, pp. 64–73.

[36] K. Rasul, C. Seward, I. Schuster, R. Vollgraf, Autoregressive denoising diffusion models for multivariate probabilistic time series forecasting, in: International Conference on Machine Learning, PMLR, 2021, pp. 8857–8868.

[37] M. Salvaris, D. Dean, W.H. Tok, Generative adversarial networks, in: Deep Learning with Azure, 2018, pp. 187–208.

[38] Y. Luo, Y. Zhang, X. Cai, X. Yuan, E00B2GAN: End-to-end generative adversarial network for multivariate time series imputation, in: Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, 2019.

[39] Y. Luo, X. Cai, Y. Zhang, J. Xu, X. Yuan, Multivariate time series imputation with generative adversarial networks, in: Neural Information Processing Systems, 2018.

[40] X. Miao, Y. Wu, J. Wang, Y. Gao, X. Mao, J. Yin, Generative semi-supervised learning for multivariate time series imputation, Proc. AAAI Conf. Artif. Intell. (2022) 8983–8991.

[41] V. Fortuin, D. Baranchuk, G. Rätsch, S. Mandt, GP-VAE: Deep Probabilistic Time Series Imputation, Cornell University, 2019, ArXiv.

[42] N. Chen, Y. Zhang, H. Zen, R. Weiss, M. Norouzi, W. Chan, WaveGrad: Estimating gradients for waveform generation, 2021.

[43] Y. Song, J. Sohl-Dickstein, D. Kingma, A. Kumar, S. Ermon, B. Poole, Score-based generative modeling through stochastic differential equations, in: International Conference on Learning Representations, 2021.

[44] J. Ho, A. Jain, P. Abbeel, Denoising diffusion probabilistic models, Adv. Neural Inf. Process. Syst. 33 (2020) 6840–6851.

[45] Y. Song, S. Ermon, Generative Modeling by Estimating Gradients of the Data Distribution, Cornell University, 2019, ArXiv.

[46] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, M. Long, TimesNet: Temporal 2D-variation modeling for general time series analysis, in: International Conference on Learning Representations, 2023.

[47] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, M. Long, Itransformer: Inverted transformers are effective for time series forecasting, in: The Twelfth International Conference on Learning Representations, 2024.

[48] S. Wang, J. LI, X. Shi, Z. Ye, B. Mo, W. Lin, J. Shengtong, Z. Chu, M. Jin, TimeMixer++: A general time series pattern machine for universal predictive analysis, in: The Thirteenth International Conference on Learning Representations, 2025, URL https://openreview.net/forum?id=1CLzLXSFNn.

[49] Z. Hu, H. Yan, Y. Zheng, H. He, C. Chen, C. Fan, K. Huang, STIOS: A novel self-supervised diffusion model for trajectory imputation in open environment scenarios, in: 2024 IEEE Smart Data, IEEE, 2024, pp. 559–566.

[50] J. Huang, Y. Xu, Q. Wang, Q.C. Wang, X. Liang, F. Wang, Z. Zhang, W. Wei, B. Zhang, L. Huang, et al., Foundation models and intelligent decision-making: progress, challenges, and perspectives, The Innovation (2025).

[51] C. Fan, L. Zeng, Y. Sun, Y.-Y. Liu, Finding key players in complex networks through deep reinforcement learning, Nature machine intelligence 2 (6) (2020) 317–324.

[52] B. Zhang, C. Fan, S. Liu, K. Huang, X. Zhao, J. Huang, Z. Liu, The expressive power of graph neural networks: a survey, IEEE Transactions on Knowledge & Data Engineering (01) (2024) 1–20.